AMENDMENT V0.3 | DESIGN ONLY

CourierLedger M1 Database Design Specification

Technical Correction Amendment v0.3

A targeted follow-up amendment to the Database Design Specification v0.1 (36 pages), as already amended by Technical Correction Amendment v0.2. This amendment supersedes only the sections, tables, constraints and test scenarios it explicitly names. Everything else in v0.1 and v0.2 stands unchanged.

> **Scope guard.** This document contains no application code and no executable migrations. SQL fragments and index definitions are illustrative design notation, marked ADAPT, exactly as in v0.1 Section E and v0.2. No courier rate, fee, statement format or contract semantic is invented. V2.2.1 formulas, integer BIGINT poisha, per-component half-up rounding, NULL versus verified zero, and the explicit D_c confirmation-plus-evidence rule are preserved without modification. CL-01 to CL-12 are preserved unless a concrete contradiction is demonstrated - and the one wording contradiction found is corrected in CL-17, with the underlying CL-01 rule unchanged.

What this amendment does

- Resolves four follow-up findings, CL-13 to CL-16, each in the established six-part form: original problem, exact change, affected section and table, constraint or transaction-level enforcement, updated test scenario, and a resolved / still-open verdict.
- Provides one implementation clarification, CL-17, on how PostgreSQL actually enforces cross-table consistency - correcting a phrase in v0.2 CL-01 without changing the CL-01 rule itself.
- Adds two tables, and only two, both under CL-16, where the existing schema is shown to be unable to hold the required history. The design total moves from 29 tables to 31.
- Ends with consolidated test changes, a traceability delta, a change log and the remaining blockers.

Reading rule. Where this amendment and v0.1 or v0.2 disagree, this amendment wins for the named item only. Where it is silent, v0.1 as amended by v0.2 applies.

> **Gating - unchanged.** M1 production implementation remains gated behind the Day 7 and Day 14 market-validation gates, merchant-confirmed courier evidence, a Docker + PostgreSQL verification environment, and explicit production-readiness approval. This amendment authorises none of those steps, and no test in it is claimed to have passed.

Baseline	Database Design Specification v0.1 (36 pages)
Prior amendment	Technical Correction Amendment v0.2 (CL-01 to CL-12)
This amendment	v0.3 - CL-13 to CL-17
Financial authority	Specification V2.2.1 and the P0 final clarification, unchanged

Amendment at a glance

Five items. Four are resolved at design level. One - CL-15 - resolves the tenant-integrity mechanics and deliberately leaves two questions open, because closing them would require a Bangladesh legal and policy decision this review does not have. CL-17 resolves a wording error in v0.2 by naming the enforcement mechanism PostgreSQL actually provides.

ID	Subject	Affected v0.1 / v0.2 locations	Verdict
CL-13	Match decision branching and concurrency: linear supersession chain	v0.2 CL-03 / CL-07; Section C (*shipment_match_decisions*); Section E append-only list	Resolved
CL-14	Idempotency recovery: abandoned claims, leases, crash windows	v0.2 CL-04; Section C (*idempotency_keys*); Section H	Resolved - lease values are operational defaults, marked ADAPT
CL-15	Workspace anonymisation and tenant integrity; shared user accounts	v0.2 CL-12; Sections C and G; OD-16	Mechanics resolved; OD-16 still open; new OD-17 opened
CL-16	Immutable bank variance history	v0.1 D.10 and *settlement_batches*; v0.2 CL-10	Resolved - two new tables, justified in the finding
CL-17	Clarification: cross-table consistency is not a CHECK	v0.2 CL-01, item 4; v0.1 Sections D and H.5	Resolved as clarification; the CL-01 rule is unchanged

What changes structurally - and what does not

- **New tables (2, both CL-16):** *settlement_variance_calculations* and *settlement_variance_calculation_inputs*. Design total: 29 tables under v0.2 becomes 31.
- **New columns:** a chain link on *shipment_match_candidates* (CL-13); lease and fencing columns on *idempotency_keys* (CL-14); an anonymisation timestamp on *merchants* (CL-15). One column is removed: the stored *bank_variance_poisha* on *settlement_batches* (CL-16), replaced by a derived value of the same name and meaning.
- **No formula changes.** Nothing in this amendment touches C, delta, V_R, S, V_{cond}, V_{total}, the identities V_{total} = C - delta - V_R + S and V_{cond} = S - V_R, or the five-value overall status model of CL-01.
- **No decision is silently closed.** OD-12 stays open. OD-16 stays open and gains the anonymisation mechanics it was waiting for. OD-17 is opened for personal data held inside append-only raw records, where anonymisation and immutability genuinely conflict.

> **A note on method for CL-13 and CL-15.** Both findings offered a tempting mechanism - merging decision branches, and moving retained records into a shared retention workspace. Both are examined in the findings below and rejected, with reasons, in favour of mechanisms that keep the existing invariants intact. Rejecting an option is part of the correction: a future implementer should not have to re-derive why the obvious-looking route was not taken.

CL-13 - Match decision branching and concurrency: one linear chain per candidate

1. Original problem

v0.2 CL-03 made match decisions immutable by moving them into the append-only table *shipment_match_decisions*, with a nullable self-reference, *supersedes_decision_id*, set when a later decision replaces an earlier one. That solved the mutability problem but left the shape of the decision history unspecified in four ways.

- **Branching is representable.** Nothing prevents two decisions from both naming the same decision as the one they supersede. The history then forks, each branch with its own latest decision, and the phrase in CL-03 - "the latest decision row for that candidate that has not itself been superseded" - identifies two rows, which may disagree: one ending in confirmed, the other in rejected.
- **Cross-candidate supersession is representable.** The v0.2 self-reference is a plain foreign key to the decision's own table and does not require the superseded decision to belong to the same candidate: a decision about candidate A can claim to supersede one about candidate B, making per-candidate derivation meaningless.
- **Concurrent first decisions have no serialisation rule.** CL-03 noted that concurrent first decisions must both be recordable so a conflict is visible, and relied on a deferred constraint trigger aimed at a different problem - a second current confirmation across candidates of the same statement line. Two reviewers deciding the same candidate at the same moment, one confirming and one rejecting, were given no rule for what happens, in what order, and what the candidate's current decision then is.
- **Cycles are not addressed.** CL-03 does not state whether a supersession cycle (A supersedes B, B supersedes A) is possible or how it is prevented. A derivation that walks a chain containing a cycle does not terminate, so the question is not academic.

The result is that the most audit-sensitive derivation in the matching design - "what is this candidate's current decision?" - was deterministic only in the easy cases.

2. Exact proposed change - definitions

Supersedes the decision-history paragraphs of v0.2 CL-03 and CL-07 where they describe derivation and concurrency. The tables and their append-only class are otherwise unchanged. Four terms are defined once:

Term	Definition
Root	A decision row for a candidate whose *supersedes_decision_id* is NULL. The first decision about that candidate.
Successor	Decision X is the successor of decision Y when X.*supersedes_decision_id* = Y.id. Y is then the *predecessor* of X.
Head	A decision that has no successor. In a well-formed history there is exactly one head per candidate.
Chain	The sequence formed by starting at the root and following successor links to the head. A well-formed history for one candidate is exactly one chain containing every decision row for that candidate.

The linear-chain rule. For each candidate, the decision history is a single linear chain: exactly one root, at most one successor per decision, and every decision belonging to the chain. Sections 4 and 5 separate how this is *prevented* from breaking, how a break is *detected* if prevention is bypassed, and how a broken history is *resolved* without editing any row.

2. Exact proposed change (continued) - prevention, derivation, conflict

(a) Prevention in normal operation - serialised inserts. Every decision insert runs in a transaction that first takes a transaction-scoped advisory lock keyed on the candidate (design notation, ADAPT: `pg_advisory_xact_lock` over a hash of the candidate ID, taken after the tenant context is set and before any decision row is read). The lock is used instead of `SELECT ... FOR UPDATE` on the candidate row for a deliberate reason: the runtime role holds no UPDATE grant on the append-only matching tables under Section E, and row-locking a table requires that privilege. An advisory lock needs no table privilege, does not modify the candidate row, is held only to commit or rollback, and cannot leak through a pooled connection because it is transaction-scoped. Within the lock, the inserting transaction reads the candidate's current head and follows one of two paths:

- If no decision exists, the new decision is inserted as the root, with *supersedes_decision_id* NULL.
- If a head exists, the new decision *must* name that head as its predecessor. A transaction that intended to record a first decision but finds, after acquiring the lock, that another decision committed first does not insert a second root: it inserts its decision as the successor of the existing head. Both reviewers' decisions are therefore preserved, in commit order, in one chain - the second decision visibly replaces the first rather than silently racing it.

The partial unique index in section 4 is the backstop: if lock discipline is ever bypassed by a code defect, a second successor insert fails with a uniqueness violation and its transaction rolls back, rather than committing a fork. A rolled-back attempt of that kind is an application error to be logged and retried through the locked path - it is not a committed decision, and nothing in this design treats it as one.

(b) Deterministic derivation of the current decision. The current decision for a candidate is derived by the following steps, in order, over that candidate's decision rows. The derivation is a pure function of the stored rows: it uses no timestamps to choose a winner, because two rows can share a timestamp and wall-clock order is not evidence.

1. If the candidate has no decision rows, it is *undecided*. Its match basis remains a proposal only, exactly as in v0.1 D.7.
2. Count roots and heads, and walk the chain from each root. If there is exactly one root, exactly one head, and the single walk visits every decision row for the candidate exactly once, the history is well-formed and the *current decision is the head*.
3. If there are multiple heads but *every* head's decision is *rejected*, the candidate is *closed by unanimous rejection*: its current decision is derived as rejected. No branch asserts a confirmation, so no branch can attach reported money to a shipment, and the disagreement among the branches has no financial consequence.
4. In every other case of a malformed history - multiple roots with disagreeing heads, or multiple heads of which at least one is confirmed - the candidate is *conflicted* and has *no current decision*. A conflicted candidate must not be used by a reconciliation run, must not satisfy the "current derived decision is confirmed" condition in CL-03, and produces a data-quality finding on any result that would otherwise have relied on it, so the shipment is classified *needs_review* under the CL-01 precedence.

(c) Conflicting initial decisions - preserved and detected, never overwritten. A conflicting pair of initial decisions can reach the table only outside the locked path: for example through a pre-v0.3 data state, or a migrator-role maintenance write. This amendment does not pretend the lock makes such rows impossible in principle; it specifies what the design does with them. They are preserved exactly as inserted - the append-only grants already make any cleanup by edit or deletion impossible for the runtime role. They are detected in two places: the derivation above returns *conflicted* rather than picking a winner, and test T-26 requires the detection to be demonstrated on deliberately malformed data written by the migrator role - a property of the derivation, not of a stored flag that could itself be wrong.

2. Exact proposed change (continued) - resolution, and why branches are never merged

(d) Resolving a conflicted candidate. Two resolution paths exist, and neither edits history:

- **Close it.** For each head that is not already a rejection, insert a successor decision of *rejected*, through the normal locked path, naming that head as predecessor and recording the conflict as the reason. When every head is a rejection, derivation step 3 applies: the candidate is closed, permanently, as rejected. This is always available and is the correct outcome when the disagreement itself makes the candidate untrustworthy.
- **Supersede the candidate, not the branch.** If the match itself is still believed correct, a new candidate row is created for the same statement line, shipment and match basis, carrying a new nullable column *prior_candidate_id* - a composite self-referencing foreign key to the conflicted candidate, same tenant - recording that this candidate replaces it. Decisions about the new candidate start a fresh, well-formed chain. The conflicted candidate and its full branching history remain stored and queryable. Nothing in Section C makes (statement line, shipment, match basis) unique, so the successor candidate is representable without weakening any existing constraint; its *rank* orders it after the candidate it replaces.

(e) Why merging branches is rejected. The alternative - a resolution decision that supersedes two heads at once, joining the branches - was considered and rejected. A single *supersedes_decision_id* column cannot express a two-predecessor join; expressing it honestly would need a link table and would change the history from a chain into a directed graph, with a derivation rule to match, for the sake of an anomaly that the locked path already prevents in normal operation. Quarantining the candidate and, where needed, superseding it keeps one simple invariant - one candidate, one chain - that every reader, query and test can rely on.

(f) The statement-line level is a separate question, with a simpler answer. CL-03's deferred constraint trigger concerned a different conflict: two *different candidates* for the same statement line both currently confirmed. That conflict needs no quarantine mechanism, because each candidate's own chain can advance independently: superseding one candidate's confirmed head with a rejection leaves exactly one currently confirmed candidate, and the line-level invariant holds again. The trigger specified in CL-03 is retained as the commit-time backstop for that invariant, and CL-17 below classifies it correctly as a deferred constraint trigger. The derivation of "the confirmed candidate for a line" is then: the unique candidate whose derived current decision is confirmed; if two qualify, the trigger has been bypassed, the line is treated as ambiguous under v0.1 D.7, and no automatic choice is made.

(g) Cycles cannot occur. The proof is short and rests on immutability rather than on any check. A cycle would require some decision row to name, directly or through a chain, itself as a predecessor. A row's *supersedes_decision_id* is set only at insert, can reference only a row that already exists (the foreign key requires it), and can never afterwards be changed, because the table is append-only with no UPDATE grant or policy for the runtime role (Section E, restated by CL-03). Every link therefore points from a later-inserted row to a strictly earlier-inserted row, so following predecessor links strictly decreases insertion order and must end at a root. No cycle can be constructed, and no cycle-detection code or constraint is needed.

3. Affected section and table

Section C, table *shipment_match_decisions* (self-reference strengthened; no new columns) and table *shipment_match_candidates* (one new nullable column, *prior_candidate_id*); Section B ERD matching sub-diagram (self-reference on candidates); v0.2 CL-03 item 4 and the CL-07 mirror wording (derivation and concurrency paragraphs superseded); Section E append-only list (unchanged membership); Section I test T-26 (new, section 5). The audit-mirror rule of CL-07 is unchanged: every decision insert still writes its *audit_logs* row in the same transaction, and the decision table remains the source of truth if the two ever differ.

4. Constraint or transaction-level enforcement

- **At most one successor per decision (no branching).** Partial unique index (ADAPT): `UNIQUE (merchant_id, supersedes_decision_id) WHERE supersedes_decision_id IS NOT NULL` on *shipment_match_decisions*. Roots are exempt by the predicate - the index governs successors, not roots - which is what allows the anomalous multi-root state of section 2(c) to be *stored and detected* rather than silently rejected by a constraint the audit then cannot see.
- **Supersession stays inside one candidate and one tenant.** Add `UNIQUE (merchant_id, candidate_id, id)` to *shipment_match_decisions* and replace the plain self-reference with a composite foreign key: `(merchant_id, candidate_id, supersedes_decision_id)` references `(merchant_id, candidate_id, id)`. A decision can now supersede only a decision about the same candidate in the same workspace, enforced by the database rather than by insert-path discipline. A CHECK keeps *supersedes_decision_id* different from the row's own *id* as a same-row guard, although the cycle proof in section 2(g) already makes self-reference unreachable in practice.
- **Serialisation.** The candidate advisory lock described in section 2(a) is a mandatory part of the decision-insert transaction, stated here as a design invariant in the same terms CL-11 used for tenant context: decision inserts pass through one component that takes the lock, and no repository method inserts a decision without it. The unique index above is the database backstop if that invariant is violated.
- **Well-formedness at commit.** Chain-shape rules that span rows - one root, chain visits every row - are cross-row invariants and are therefore *not* expressed as CHECKs (see CL-17). They are verified by a deferred constraint trigger (ADAPT) on *shipment_match_decisions* that, at commit, re-derives the candidate's state using section 2(b) and rejects the transaction if it would leave a *newly written* history malformed through the application's own path. Rows written by the migrator role outside that path are the detection case of T-26, not a state the trigger is asked to repair: triggers reject transactions; they do not rewrite history.
- **Immutability unchanged.** No UPDATE or DELETE policy or grant is added for either matching table. The lock, the indexes and the trigger all operate on inserts only.

5. Updated test scenario

T-26 (new) - Match decision chain integrity and concurrency. (a) Through the locked path, confirm a candidate, then supersede with a rejection, then supersede with a confirmation: exactly one chain of three rows exists, the derived current decision is the final confirmation, and T-21's assertions still hold. (b) Two concurrent transactions decide the same undecided candidate, one confirming and one rejecting: both decisions are committed, in one chain, in commit order; the derived current decision is the second transaction's decision; no second root exists. (c) Attempt, bypassing the application path as the migrator role, to insert a second successor of an existing decision: rejected by the partial unique index. (d) Attempt to insert a decision superseding a decision belonging to a different candidate, and separately to one in a different workspace: both rejected by the composite foreign key. (e) Insert two roots for one candidate as the migrator role, one ending confirmed and one rejected: the derivation reports the candidate conflicted with no current decision, a run that would rely on it classifies the shipment *needs_review* with a data-quality finding, and neither row has been altered. (f) Close the conflicted candidate by rejecting every non-rejected head: the derivation now reports rejected under the unanimous-rejection rule. (g) Create a successor candidate linked by *prior_candidate_id*, confirm it through the locked path, and confirm the run snapshot records the new candidate's decision row ID while the conflicted candidate's history remains fully queryable.

6. Status

Resolved. The decision history is now a defined linear chain: branching is prevented by a unique index, cross-candidate links are prevented by a composite foreign key, concurrent inserts are serialised by a transaction-scoped lock chosen to respect the append-only grant model, current-decision derivation is deterministic in every state including malformed ones, cycles are shown to be structurally impossible, and conflicts are preserved, detected and resolvable without a single UPDATE or DELETE. Immutable decision history - the point of CL-03 and CL-07 - is not weakened; it is what makes the cycle proof and the conflict-preservation rule possible.

CL-14 - Idempotency recovery: abandoned claims, leases and crash windows

1. Original problem

v0.2 CL-04 established one authoritative idempotency store, *idempotency_keys*, with a lifecycle of *in_progress*, *completed* and *failed*: the key row is claimed in its own short transaction, the work transaction on success marks it completed and sets the typed result reference, and a rolled-back work transaction leaves the key to be marked failed by a compensating update so that a same-hash retry may re-execute. That design answers replay and hash-conflict questions, but it leaves the failure it most needs to survive - a process that stops mid-operation - unspecified in three ways.

- **A crashed claim is indistinguishable from a running one.** If the worker crashes after claiming a key, or crashes before the compensating failed-marking runs, the row sits at *in_progress* indefinitely. CL-04 gives no rule for when another worker may treat the claim as abandoned. Every later retry with that key then faces a row that is neither completed (so there is no result to return) nor failed (so re-execution is not authorised). The operation is wedged - safely, but permanently.
- **Reclaiming without ownership invites the duplicate the table exists to prevent.** The obvious fix - "let a retry take over a stale in-progress row" - creates the opposite defect if it is not fenced: the original worker may not be dead, only slow or partitioned. If a second worker re-executes while the first is still working, and the first then completes, two import batches or two runs can be created for one key. For run creation, CL-05's *UNIQUE (merchant_id, run_hash)* would still collapse identical inputs to one run; for import creation there is no equivalent content-level backstop stated. An unfenced takeover is therefore not an acceptable recovery rule.
- **The crash window between work and key completion is unexamined.** CL-04's wording puts the completion update in "the work transaction", but does not state the requirement strictly enough to reason about: if the work and the completion were ever in separate transactions, a crash between them would leave a committed result that no key points to, and a retry would create a second one. Whether that window exists is a property of the transaction boundary, and the amendment must close it explicitly rather than assume it away.

2. Exact proposed change - lease, token and attempt columns

Clarifies and extends v0.2 CL-04, item 2 and item 4. The table remains the sole idempotency owner; the operation list, typed result references, uniqueness and request-hash rules are unchanged. Four columns are added to *idempotency_keys*:

Column	Type / null	Meaning
`lease_expires_at`	TIMESTAMPTZ, NULL	End of the current claim's ownership window. Set at every claim and reclaim; NULL on completed and failed rows. A claim is *live* while this is in the future and *abandoned* once it has passed.
`claim_token`	UUID, NULL	A fresh random token generated at every claim and reclaim - the fencing token for this attempt. Only the holder of the current token may complete or fail the key. Holds no credential and no user information.
`attempt_count`	INTEGER, NOT NULL, DEFAULT 0, CHECK >= 0	Number of times the key has been claimed. Incremented atomically by the claim or reclaim that wins. Bounds recovery work: see the attempt cap in section 2(g).
`claimed_by_instance`	TEXT, NULL	Opaque identifier of the application instance holding the claim, for operational diagnosis only. Not an actor identity, not a credential, and never used for authorisation - the token, not the label, fences the claim.

Two different clocks, never conflated. CL-04's *expires_at* remains the *replay horizon*: how long a completed result may be returned for a replayed key, after which the row may be purged by the maintenance role and the key string claimed fresh. The new *lease_expires_at* is the *ownership horizon*: how long one worker may hold an unfinished claim. The lease is measured in minutes; the replay horizon is the 24-hour design default set by CL-04. Reclaiming a lease never extends *expires_at*, and *expires_at* passing never, by itself, authorises re-execution of an operation whose status is completed - it ends the replay guarantee, which is a different statement, addressed in item (f) below.

2. Exact proposed change (continued) - claim, reclaim, completion

(a) Claim. As in CL-04, the key row is inserted in its own short transaction, now with status *in_progress*, *attempt_count* = 1, a fresh *claim_token* and *lease_expires_at* set to the claim time plus the operation's lease duration. Lease durations are operational configuration, not business rules: the design default is 5 minutes for *import_create* and 10 minutes for *run_create* (ADAPT - to be set from measured operation times in the verification environment, and always comfortably longer than the operation's normal duration and far shorter than the replay horizon). A worker that is still legitimately working as its lease nears expiry may extend the lease by a conditional update that matches its own current *claim_token*; a worker whose token no longer matches cannot extend, complete or fail the key, under any circumstances.

(b) Reclaim of an abandoned claim. A retry that finds its key at *in_progress* with an expired lease attempts exactly one atomic, conditional update (design notation, ADAPT): set a fresh *claim_token*, a new *lease_expires_at*, increment *attempt_count* and record the reclaiming instance, *where* the row's key, operation and *request_hash* match the retry, its status is *in_progress*, and its lease has expired. The database executes that update against at most one row, so among any number of concurrent reclaimers exactly one wins; the losers observe the new live lease and behave as ordinary concurrent retries - they wait within a bounded period or return the operation's in-progress state to the caller, and they do not execute the work. A retry whose hash differs from the stored hash is a conflict error in every state of the row - live, abandoned, failed or completed - exactly as CL-04 requires; abandonment never launders a hash conflict into a fresh execution.

(c) Completion is in the work transaction - the crash window is closed, not managed. This amendment makes explicit what CL-04 implied: the update that marks the key *completed* and sets its typed result reference is the *last write of the same transaction that creates the result*, and that update is conditional on the row still carrying the worker's *claim_token* and status *in_progress*. If the conditional update affects zero rows - because the lease was reclaimed and the token rotated - the work transaction must roll back in full, discarding the result it created. The consequences are strict: a result and its key completion commit together or not at all, so no committed result can exist without its key pointing at it; and a displaced ("zombie") worker cannot commit duplicate work, because its completion is the one write it is no longer able to make, and that write gates its whole transaction.

(d) Failure. The CL-04 compensating update is retained and fenced in the same way: a worker that knows its work has rolled back marks the key *failed*, conditional on its own token, clearing the lease. A *failed* key may be re-claimed by a same-hash retry immediately, without waiting for any lease. If the worker crashed before it could mark failure, the lease expiry path of (b) reaches the same outcome more slowly. Both paths converge on one rule: re-execution is authorised either by a failed row or by an expired lease - never by the mere passage of an arbitrary wait, and never by a second live claim.

The four crash windows, stated individually

Window	Outcome under this design
W1. Crash after the claim commits, before work begins	No result exists; the key is *in_progress* under a lease that will expire. A same-hash retry then reclaims under (b) and executes the work once.
W2. Crash during the work transaction	The work transaction rolls back, so no partial result exists. Recovery proceeds as W1: lease expiry, reclaim, one execution.
W3. Crash after work commits, before the caller receives the response	There is no "after work commits but before the key completes" state, by rule (c): they are one commit. The retry finds a *completed* key and receives the original typed result reference. No re-execution.
W4. Crash before the compensating failed-marking	The key remains *in_progress* until its lease expires, then follows (b). Slower than an explicit failure, identical in outcome.

2. Exact proposed change (continued) - replay, expiry and the attempt cap

(e) Completed replay is unchanged, and total. A retry with the same key, operation and hash against a *completed* row returns the typed result reference and executes nothing, at any point before *expires_at* - including a retry that arrives after one or more abandoned-claim episodes earlier in the key's life. The history of claims is operational detail; the completed row is the answer.

(f) What expiry does and does not end. After *expires_at*, CL-04 permits the maintenance role to purge the key row and the same key string to be claimed fresh - that is, the deduplication guarantee for that key string ends. Two existing safeguards bound the risk, and this amendment relies on them rather than inventing a third: for *run_create*, CL-05's *UNIQUE (merchant_id, run_hash)* deduplication is content-based and has no expiry, so a post-expiry duplicate run request still resolves to the one existing run; for *import_create*, the file-level hash detection in Section C (*file_objects* hash per merchant) is the corresponding second layer, and an import retry after key expiry must be checked against it before a new batch is created. What expiry must never do is delete or alter the created batch or run themselves - CL-04 already states this, and it is repeated here because recovery logic is exactly where such a cleanup would be attempted.

(g) Attempt cap. Recovery must not loop forever on an operation that crashes every worker that touches it. When a claim or reclaim would raise *attempt_count* above the operation's configured cap (design default 5, ADAPT), the row is instead moved to *failed*, its lease cleared, and further retries with the same key return an explicit terminal-failure response directing the caller to investigate and, if appropriate, submit under a new key. The cap is enforced inside the same conditional update, so concurrent reclaimers cannot race past it. A capped key is a visible operational event, recorded in the application log outside the tenant tables in the manner of CL-11 - a repeatedly crashing financial operation is something operators must see, not something the table should quietly absorb.

3. Affected section and table

Section C, table *idempotency_keys* (four columns added; status list unchanged - *abandoned* is a derived condition, *in_progress* with an expired lease, not a fourth status, so no CHECK list changes and no stored flag can disagree with the clock); Section H (the shared idempotency component of CL-04 gains the claim, reclaim and completion protocol above); Section I test T-10 (assertions extended by T-27, section 5). Sections C and H of v0.1 are affected only through the CL-04 supersession chain. No other table changes: in particular, no lease or token column appears on *import_batches* or *reconciliation_runs*, preserving CL-04's single-owner rule.

4. Constraint or transaction-level enforcement

- **Uniqueness remains the arbiter of first claim.** *UNIQUE (merchant_id, operation, idempotency_key)* is unchanged; the losing claimant of a race sees the existing row and follows the reclaim, wait or replay path - no check-then-insert in application code is relied on, as CL-04 already required.
- **Reclaim and completion are single conditional updates.** Each is one UPDATE whose predicate includes the expected status, the lease condition where relevant, and - for completion and failure - the caller's *claim_token*. PostgreSQL's row locking makes each such update atomic against concurrent updaters of the same row: exactly one reclaim wins, and a completion by a displaced token matches zero rows. These are transaction-level mechanisms, stated as such; no CHECK constraint is claimed to perform them (see CL-17).
- **State-to-column consistency is row-local and is a CHECK.** *completed* requires *completed_at* NOT NULL, the operation's typed result reference NOT NULL (per CL-04's existing CHECK) and *lease_expires_at* NULL; *in_progress* requires *lease_expires_at* NOT NULL and all result references NULL; *failed* requires *lease_expires_at* NULL and all result references NULL. Each condition inspects only the row itself, which is precisely what a CHECK may do.
- **RLS and roles unchanged.** The table remains tenant-owned, RLS ENABLE + FORCE, with the single sanctioned UPDATE identified in CL-04 - now understood to be exactly the claim, reclaim, completion and failure updates above, all tenant-scoped and predicate-guarded. Purge after expiry remains a maintenance-role operation, never a runtime grant.

5. Updated test scenario

T-27 (new) - Idempotency recovery. T-10's assertions (a) to (g) are retained, then: (h) Claim a key, simulate a crash before work begins (W1), advance past the lease, and retry with the same hash: the retry reclaims, *attempt_count* becomes 2, the token differs from the original, and exactly one result is created. (i) Repeat the crash during the work transaction (W2): no partial batch or run exists at any point. (j) Commit the work, then simulate the caller never receiving the response (W3): the retry returns the original result reference and creates nothing. (k) Zombie fencing: reclaim an apparently abandoned key, then have the original worker attempt its conditional completion with the old token: it matches zero rows, its transaction rolls back, and exactly one result exists. (l) Race two reclaimers against one expired lease: exactly one wins; the loser executes nothing. (m) Attempt to reclaim or replay an abandoned key with a different hash: conflict error, no execution, in every row state. (n) Drive a key to the attempt cap with repeated crashes: the row ends *failed*, further same-key retries return the terminal-failure response, and a submission under a new key succeeds normally. (o) After a key expires and is purged, resubmit an identical run request under a fresh key: CL-05 deduplication still returns the single existing run.

6. Status

Resolved at design level. Every state a key row can be left in by a crash now has a defined, fenced recovery path, and the no-duplicate guarantee rests on mechanisms - unique constraint, atomic conditional updates, token fencing and same-transaction completion - rather than on workers being well-behaved. The lease durations and attempt cap are operational defaults marked ADAPT; setting their final values from measured behaviour in the verification environment is implementation tuning, not an open design decision, and does not join the blocker list.

CL-15 - Workspace anonymisation and tenant integrity: the tombstone merchant

1. Original problem

v0.2 CL-12 specified a two-stage model - closure, then erasure - and said financial history would be "retained in anonymised-tenant form - merchant identity link severed, amounts and rule snapshots intact". That sentence contains an unspecified mechanism, and each obvious way of implementing it breaks a different part of the design:

- **Deleting the merchant row** is rejected by the database itself: every financial table references *merchants* through foreign keys that CL-12 confirmed as ON DELETE RESTRICT. The row cannot go while history exists - which is the point of the restriction.
- **"Severing the identity link"** by rewriting *merchant_id* on retained rows would require UPDATEs across runs, results, findings, fee lines, statement lines and settlement records - tables whose defining property is that the runtime role cannot update them at all, and whose snapshots are supposed to be the historical fact. It would also falsify those rows: a run would claim to belong to a workspace it never belonged to.
- **Anonymising only the merchant's name** leaves the people unanswered. A workspace's members are *users* rows, and a user is not owned by a workspace: the same person may hold memberships in several workspaces. CL-12 said "user personal fields are anonymised" in the context of one workspace's erasure, without stating what happens when that user is an active member of another workspace that is not closing. Taken literally, closing one workspace could erase the sign-in identity a person still uses daily in another.
- **RLS has no stated post-anonymisation behaviour.** Retained rows carry a *merchant_id* that no living session can legitimately assert once memberships have ended, and CL-12 did not state what that means for reachability or lawful export.

2. Exact proposed change - the tombstone merchant record

Clarifies v0.2 CL-12 Stage 2 and Section G. The central decision is this: **the anonymised retention workspace is the same merchant row, anonymised in place.** Anonymisation is an operation on identity-bearing fields of mutable identity-class rows. It is never an operation on *merchant_id*, and it never touches a financial row. Stage 2 for a closed workspace performs, in one migrator-role maintenance transaction:

- **On *merchants*:** *name* is replaced with a non-identifying label of the form "Anonymised workspace" plus an opaque token that encodes no personal data; *trading_name* is set NULL; the new column *anonymised_at* (TIMESTAMPTZ, NULL) is set. The row's *id* - the value every *merchant_id* in the database references - is unchanged, and *status* remains *closed*. Anonymising the business name is mandatory, not optional: for a sole proprietor the trading name may itself be personal data, and a design that scrubs the person but keeps the shop name has not anonymised anything.
- **On *merchant_courier_accounts* of that workspace:** *merchant_account_ref* is set NULL and *display_label* replaced or nulled where it contains identifying text. Courier account references are the merchant's identifiers with third parties and have no place in retained history; the internal courier identity survives through *courier_id*, which is global reference data and identifies no merchant.
- **On *file_objects* and stored bytes:** as CL-12 already specified - bytes are purged, *purged_at* is set, and the metadata row with its hash remains so historical references still resolve.
- **On financial and append-only tables:** nothing. No UPDATE, no DELETE, no *merchant_id* change. Runs, results, findings, fee lines, statement lines, import rows, allocations and - under CL-16 below - variance calculations remain byte-identical, with amounts, snapshots and rule provenance intact.

The result is a *tombstone merchant*: a tenant root that still exists for every foreign key that needs it, still scopes its retained history under its own original *merchant_id*, and no longer identifies anyone.

2. Exact proposed change (continued) - options rejected, and the shared user rule

Why not a shared anonymised retention workspace. The alternative considered - one special "retention" merchant to which every closed workspace's financial rows are transferred - is rejected for four concrete reasons. First, the transfer is the forbidden rewrite of *merchant_id* on append-only rows described above. Second, it merges tenants: histories of unrelated businesses would come to share one *merchant_id*, ending the one-workspace-one-tenant correspondence the whole of Section E rests on. Third, it corrupts scoped uniqueness: run deduplication is *UNIQUE (merchant_id, run_hash)* (CL-05), and two transferred workspaces could present the same hash under the retention tenant and collide. Fourth, it makes isolation unprovable for the retained data, because an RLS context asserted for the retention tenant would expose every transferred workspace at once. The tombstone keeps each workspace's history under its own key, which is what "tenant isolation for retained records" has to mean.

The shared user rule. User anonymisation is governed by the user's position across *all* workspaces, never by one workspace's closure alone. The rule is deterministic:

1. When a workspace is anonymised, its *merchant_memberships* rows - already ended at closure under CL-12 Stage 1 - are left as historical records of role and period. A membership row carries no personal fields beyond its foreign keys, so it needs no scrubbing; the identity question is entirely about the *users* row it points to.
2. A *users* row is anonymised - *email* replaced with a non-routable anonymised placeholder unique to that row, *display_name* set NULL, credential material removed, *status* set to *closed* - *only if* the user has no membership in *active* or *invited* state in any workspace, and no live session. The eligibility test is evaluated globally at the moment Stage 2 runs, not inferred from the closing workspace.
3. If the user has any active membership elsewhere, their *users* row is not touched in any way by this workspace's anonymisation. They keep signing in; the other workspace's audit attributions keep resolving; nothing about them is recorded in the closing workspace's anonymisation audit beyond the fact that a member's identity was retained because of other memberships - a fact recorded without naming the other workspace.
4. Where a *users* row is anonymised, it is anonymised *in place* and the row remains: *actor_user_id*, *created_by_user_id*, *decided_by_user_id* and similar attributions across retained history continue to reference an existing row, preserving referential integrity and provenance, while the row no longer identifies a person. A UUID that resolves to an anonymised user is a pseudonymous attribution, and the design says so plainly rather than claiming the attribution has been erased.

Worked example. A person is Owner of workspace A and a member of workspace B, both active. A closes and completes Stage 1. At Stage 2, A's tombstoning proceeds in full - business identity scrubbed, courier references nulled, file bytes purged - but the eligibility test finds an active membership in B, so the *users* row is untouched and the person signs in to B the next day exactly as before. If B later closes and is anonymised in its turn, the test then finds no active membership anywhere, and only at that point is the *users* row anonymised.

2. Exact proposed change (continued) - isolation and export after anonymisation

Tenant isolation is preserved, and is in fact at its strongest. Retained rows keep their original *merchant_id*; RLS remains ENABLE + FORCE on every tenant table; and tenant context can only be set from an authenticated session's resolved *active* membership (CL-11). A tombstoned workspace has no active memberships, so no application session can legitimately assert its context: through the runtime role the retained history is unreachable, while remaining physically intact and internally consistent. Isolation between two tombstoned workspaces is the same isolation as between two live ones - distinct keys, distinct policy scope - because anonymisation changed no key.

Historical export has two windows, and they must not be confused. The merchant-facing export - the owner's complete record of their own history - belongs to Stage 1: the workspace is closed but not yet anonymised, still readable by its former Owner for export, as CL-12 specified. That is the only window in which an identified export exists, and the length of the grace window before Stage 2 remains the open policy question in OD-16. After anonymisation, the only exports the design permits are anonymised ones, produced under the migrator-role maintenance procedure rather than the application: amounts, dates, statuses, findings and rule snapshots keyed by the tombstone's *merchant_id*, containing no merchant or user identity. Such an export must exclude the raw source payloads discussed immediately below.

> **The one conflict this amendment does not resolve: personal data inside append-only raw records.** Anonymising identity rows does not reach personal data that may exist *inside* retained source data. *import_rows.raw_values* preserves each uploaded row verbatim, by design, and courier statement files may contain recipient names, phone numbers or addresses as supplied in the source file; statement lines and order references can carry merchant-supplied identifiers with similar content. *orders* are mutable-class rows whose identifying fields can be scrubbed in Stage 2. But *import_rows* and *statement_lines* are append-only precisely so that raw and normalised values can never be rewritten (v0.1 D.3). Scrubbing them would break the raw-preservation invariant; deleting them would break referential integrity for the runs built on them; retaining them unchanged means retained history can still contain third-party personal data after the merchant's own identity is gone. This amendment does not pick a winner among anonymisation, immutability and retention. The question is opened as OD-17 for the same Bangladesh legal and policy review as OD-16, with the interim design position stated honestly: until OD-17 is decided, retained raw records of a tombstoned workspace are sealed - unreachable through the runtime role, excluded from anonymised exports, and accessed only under the break-glass procedure contemplated in Section E.5, which is itself unauthorised until production-readiness approval.

3. Affected section and table

Section G (workspace deletion and user deletion subsections - the CL-12 two-stage model gains the mechanics above); Section C, table *merchants* (one column added, *anonymised_at*; the v0.1 *closed_at* column and status list are unchanged); Section C, tables *users* and *merchant_courier_accounts* (Stage 2 field treatment specified); Section J (OD-16 clarified, new OD-17); Section I test T-25 (revised) and T-30 (new). No financial table is affected in any way - which is the point of the design.

4. Constraint or transaction-level enforcement

- **Referential integrity needs no new mechanism - it is never put at risk.** Because no row is deleted and no key value changes, every existing foreign key, composite tenant key and ON DELETE RESTRICT rule continues to hold before, during and after anonymisation. The hard DELETE rejection asserted in T-25 remains the database's guarantee that this path is the only path.
- **Stage 2 is one maintenance transaction per workspace, under the migrator role only.** It updates exactly the identity-bearing mutable rows listed in section 2, sets *anonymised_at*, and writes its audit record - recording which categories of field were anonymised and when, never the erased values, as CL-12 required. The runtime role receives no new grant of any kind: a closed workspace's INSERT-blocking write policies (CL-12) remain in force, and anonymisation is not an application operation.
- **Closed means closed, permanently, at the database.** Once *anonymised_at* is set, the workspace cannot be re-opened: the CL-12 grace-window re-open applies only between closure and anonymisation. This is enforced by the same status-and-policy pattern as closure (ADAPT) and asserted in T-25.
- **User eligibility is a transaction-level rule.** The global-membership test spans tables and therefore cannot be a CHECK (see CL-17); it is performed inside the Stage 2 maintenance transaction, which takes the relevant membership rows under lock so a concurrent invitation or reactivation of the same user in another workspace cannot race the anonymisation, and it is backstopped by the T-30 assertions.

5. Updated test scenario

T-25 (revised) - Workspace closure, tombstoning and retention. CL-12's assertions are retained (closure blocks new imports and runs at the database level; historical runs remain readable and byte-identical; hard DELETE of the merchant row is rejected), then Stage 2 is executed. Expected: *merchants.name* and *trading_name* no longer identify the business; *anonymised_at* is set; courier account references are NULL; file bytes are purged with *purged_at* set; every financial row is byte-identical to its pre-anonymisation state, including *merchant_id*; no reconciliation identity (V_{total} = C - delta - V_R + S) is affected on any retained result; a runtime-role session can read none of the tombstoned workspace's rows; and re-opening the workspace is rejected.

T-30 (new) - Shared user across workspaces. Create user U as Owner of workspace A and active member of workspace B, with audit-attributed actions in both. Close and anonymise A. Expected: U's *users* row is unchanged - email, display name and credential material intact; U can still authenticate and B's data is fully accessible under B's context; A's retained audit rows still resolve their actor reference to U's row; A's tombstone is complete. Then close and anonymise B, leaving U with no active membership anywhere: the *users* row is now anonymised in place, the row itself still exists, and historical actor references in both workspaces still resolve - to an anonymised user, not to a dangling key and not to an identifiable person.

6. Status

Partially resolved - mechanics resolved, two policy questions open. The apparent conflict between retention, anonymisation, mandatory tenant keys, RLS and shared accounts is resolved at the level this document owns: nothing has to move, merge or be rewritten for a workspace to stop identifying anyone while its financial history stays intact, isolated and exportable in anonymised form, and no shared user can lose their identity to another workspace's closure. What remains open is not mechanics: the grace-window length, the financial retention period and the erasure-versus-retention conflict rule stay in OD-16, and the treatment of third-party personal data inside append-only raw records is opened as OD-17. Both require Bangladesh legal and policy review; both block production launch, not design review.

CL-16 - Immutable bank variance history: a calculation record, not a column

1. Original problem

In v0.1, bank variance lives in exactly one place: *settlement_batches.bank_variance_poisha*, a single nullable BIGINT column on a table whose status and totals are otherwise mutable. v0.2 CL-10 then defined what a reversal does to a computed variance - the batch "no longer reports a current bank variance (reported as NULL) while the historical computation remains in the audit record", and a re-completion "computes a fresh variance". Taken together, the two documents require a history that the schema cannot hold:

- **One column cannot store a sequence.** The first variance is overwritten by the fresh variance after re-allocation, or cleared towards NULL after a reversal. Either way the earlier computed value - a financial figure the merchant may already have seen and acted on - is gone from structured storage at exactly the moment a dispute would ask for it.
- **The audit record is not a substitute.** CL-10's fallback is *audit_logs.detail*, a JSONB payload. It has no typed amount column, no foreign key to the receipts or allocations the calculation used, no uniqueness or sequence, and no constraint that a variance-shaped entry even exists for a given computation. Reconstructing "what variance was shown on this date, from which receipts and allocations" becomes a search through unstructured log text - the outcome this finding was raised to prevent.
- **The inputs are not snapshotted anywhere.** A variance is a function of three things that can all change afterwards: the expected batch payout, the set of receipts and their completeness, and the net allocation total (which reversals change, by design). Even if each successive variance value were kept, a value without its inputs cannot be verified, and an unverifiable historical financial figure is little better than none.

The task for this amendment permitted a new table only if the existing schema cannot safely support the requirement. That test is met, and section 2 says why in one sentence: the only existing stores are a single mutable column, which cannot hold a sequence, and an unstructured audit payload, which cannot hold a verifiable calculation. Two tables are therefore added - one for the calculation, one for its typed input references - and no third: amounts, inputs and supersession can all be expressed with exactly those two.

2. Exact proposed change - *settlement_variance_calculations*

Supersedes the bank variance parts of v0.1 Section C (*settlement_batches*) and D.10, and the variance-history sentence of v0.2 CL-10. Bank variance is henceforth an append-only calculation record. *settlement_batches* keeps every other column and rule unchanged - expected payout, allocation status, bank status - but its stored *bank_variance_poisha* column is **removed from the design**; the name survives as a derived value with identical meaning, defined below.

settlement_variance_calculations - one row per computation of bank variance for one settlement batch. Tenant-owned. RLS enforced. Append-only. Principal columns:

Column	Type / null	Constraints and meaning
`id`	UUID, NOT NULL	PK; UK part of (merchant_id, id)
`merchant_id`	UUID, NOT NULL	FK to merchants(id)
`settlement_batch_id`	UUID, NOT NULL	Composite FK to settlement_batches
`calculation_seq`	INTEGER, NOT NULL	CHECK >= 1. Sequence within the batch, assigned in the computing transaction. UNIQUE (merchant_id, settlement_batch_id, calculation_seq)
`expected_batch_payout_poisha`	BIGINT, NOT NULL	Snapshot of the batch's expected payout at computation time. NOT NULL by precondition: no calculation row exists while the expected payout cannot be calculated exactly
`allocated_bank_confirmed_total_poisha`	BIGINT, NOT NULL	Snapshot of the net allocated bank-confirmed total at computation time - allocations minus their reversals, on the CL-10 net-total rule
`bank_variance_poisha`	BIGINT, NOT NULL	The computed variance: allocated bank-confirmed total minus expected batch payout. Signed; a negative variance is a legitimate arithmetic result, is not the negative-payout question of OD-12, and is never labelled a confirmed loss
`computed_at`	TIMESTAMPTZ, NOT NULL	DEFAULT now()
`computed_by_user_id` / `actor_type`	UUID NULL / TEXT NOT NULL	CHECK in ('user','system'), following the audit_logs pattern: a computation triggered by a merchant completing a receipt records that user; a system-triggered recomputation records actor_type 'system' with a NULL user

2. Exact proposed change (continued) - inputs, current value, supersession

settlement_variance_calculation_inputs - the typed reference set for one calculation. Tenant-owned, RLS enforced, append-only:

Column	Type / null	Constraints and meaning
`id`	UUID, NOT NULL	PK
`merchant_id`	UUID, NOT NULL	FK to merchants(id)
`calculation_id`	UUID, NOT NULL	Composite FK to settlement_variance_calculations
`input_kind`	TEXT, NOT NULL	CHECK in ('receipt','allocation')
`bank_receipt_id`	UUID, NULL	Composite FK to bank_receipts. NOT NULL exactly when input_kind = 'receipt'
`settlement_allocation_id`	UUID, NULL	Composite FK to settlement_allocations. NOT NULL exactly when input_kind = 'allocation'. Reversal rows are allocations under CL-10 and are listed as inputs in their own right

The exactly-one-reference rule is a row-local CHECK - the same XOR pattern CL-04 established for typed idempotency references - so every input is a real, tenant-checked foreign key, never a bare identifier in a text or JSON field. The receipt set listed is *every* receipt linked to the batch at computation time, because the completeness precondition is a statement about all of them. The calculation and its complete input set are inserted in the transaction that establishes its preconditions; a partial input set is not a state this design permits to commit (see section 4).

Preconditions for a calculation row to exist at all - restated from v0.1 D.10 and v0.2 CL-10 as insert conditions: the expected payout is NOT NULL; every linked receipt is explicitly marked complete; and the batch is fully allocated on net totals. Until all three hold, there is no calculation row and the current variance is NULL.

Current variance is derived, never stored. The value presented for a batch - still named *bank_variance_poisha* at the read surface - is: the *bank_variance_poisha* of the batch's highest-*calculation_seq* calculation, *if and only if* that calculation's preconditions still hold against current data: the batch's expected payout and current net allocated total still equal its snapshots, and every currently linked receipt is complete. In every other case the current variance is NULL. This is CL-10's rule, now resting on a structured comparison instead of a presentation convention.

Supersession after reversal. A reversal changes no calculation row: it makes the latest calculation fail the still-holds comparison, so current variance becomes NULL, while the history remains queryable in sequence. Re-allocation to full allocation triggers a *new* calculation row with the next *calculation_seq*, snapshotting the new net total and the then-current allocation set, including the reversal row. Supersession is derived from sequence order: there is deliberately no *is_current* flag, because a flag on an earlier row would have to be updated when a later calculation is inserted, and these rows are never updated The CL-10 audit row is still written for each computation and reversal, as a mirror: where CL-10 said history "remains in the audit record", this amendment corrects the emphasis - history is in the calculation table, and the audit record mirrors it, as CL-07 established for match decisions.

3. Affected section and table

Section C (two new tables - 31 in total, up from 29 under v0.2; *settlement_batches* loses its stored *bank_variance_poisha* column); Section B ERD settlement sub-diagram; Section D.10 (variance definition restated); Appendix 1 bank-variance row (enforcement reference updated); v0.2 CL-10 (variance-history sentence superseded); Section I test T-28 (new) and T-17 (strengthened, section 5).

4. Constraint or transaction-level enforcement

- **Append-only, like every other financial history table.** Both new tables join the Section E append-only class: RLS ENABLE + FORCE, SELECT and INSERT policies and grants only for the runtime role, no UPDATE or DELETE policy or grant. A variance, once computed and shown, cannot be edited into a different number; it can only be succeeded.
- **Row-local CHECKs.** The variance arithmetic is same-row and is therefore a legitimate CHECK: *bank_variance_poisha* = *allocated_bank_confirmed_total_poisha* - *expected_batch_payout_poisha*. The inputs table carries the XOR CHECK on its two reference columns and the *input_kind* list. *calculation_seq* is positive and unique per batch by its unique constraint.
- **Tenant integrity of inputs.** Every reference is a composite (merchant_id, id) foreign key, so a calculation in one workspace cannot list a receipt or allocation from another, and a calculation cannot be attached to another workspace's batch - the same guarantee CL-04's typed references provide for idempotency.
- **Preconditions and input completeness are transaction-level.** Whether all linked receipts are complete and the batch is fully allocated is a statement about other tables and about sets of rows; it cannot be a CHECK (see CL-17). The computing transaction therefore runs under the same row lock on the settlement batch that CL-10's allocation protocol already takes - which serialises calculation against concurrent allocation and reversal for that batch - verifies the three preconditions, derives the input set, and inserts the calculation and its inputs atomically. A deferred constraint trigger (ADAPT) is the commit-time backstop: it rejects a committed calculation whose snapshots do not equal the precondition values derivable from its listed input set, so an insert path that skipped the verification cannot leave a false calculation standing.
- **Sequence integrity.** *calculation_seq* is assigned as one more than the batch's current maximum, inside the batch-locked transaction, so concurrent computations for one batch cannot interleave or reuse a sequence value; the unique constraint is the backstop, as with CL-13's successor index.
- **Tombstone compatibility.** Both tables are financial history under CL-15: anonymisation never updates or deletes them, their *merchant_id* continues to scope them under the tombstoned workspace, and an anonymised export draws variance history from these tables rather than from audit payloads.

5. Updated test scenario

T-28 (new) - Bank variance history. Build a batch with two partial receipts (T-17's fixture): at every intermediate step, expected - no calculation row exists and current variance is NULL. Complete both receipts and fully allocate: exactly one calculation row exists, seq 1, its input set lists both receipts and every allocation row, its variance equals the snapshotted totals' difference, and current variance equals it. Reverse one allocation (CL-10): the seq-1 row is byte-identical, no new calculation exists, and current variance is NULL. Re-allocate to full allocation with a different net total: a seq-2 calculation exists with the new total and an input set that includes the reversal row; current variance equals seq 2; seq 1 remains queryable. Then: an UPDATE or DELETE on either new table as the runtime role affects zero rows or is denied; a calculation insert attempted while a receipt is incomplete is rejected; an input row referencing another workspace's receipt is rejected by the composite foreign key; and a hand-inserted calculation whose snapshotted total disagrees with its listed inputs is rejected at commit by the deferred trigger. **T-17 is strengthened** accordingly: its phrase "the historical computation remains auditable" now asserts the seq-1 calculation row and its inputs, not an audit payload.

6. Status

Resolved. Historical bank variance now has a structured, immutable, verifiable home: value, inputs, actor and time are relational records with foreign keys and constraints, supersession after reversal is a derivation rather than an edit, and the current-variance rule of v0.1 D.10 and CL-10 is preserved exactly. The new-table condition in the commissioning brief is addressed head-on in section 1: the existing schema offered a single mutable column and an unstructured log, and neither can safely hold this history.

CL-17 - Clarification: cross-table consistency is not a CHECK constraint

1. Original problem

v0.2 CL-01, item 4, specified the enforcement of the corrected status rule in these words: "A consistency CHECK (or deferred constraint trigger, ADAPT at implementation) rejects the impossible combination overall_status = 'matched' together with any stored finding on the same result whose amount is non-zero outside the run tolerance." The parenthetical treats two different mechanisms as interchangeable options for one rule. They are not interchangeable, and for this rule only one of them exists.

The rule inspects three tables: the result's *overall_status* sits on *reconciliation_results*; the findings and their amounts sit on *reconciliation_findings*; the tolerance they are compared against sits on *reconciliation_runs*. A PostgreSQL CHECK constraint is evaluated against the single row being inserted or updated in its own table. It cannot contain a subquery, cannot reference another table, and cannot reference another row - PostgreSQL rejects such definitions at creation time. There is therefore no CHECK, on any of the three tables, that can express "matched while a live finding exists". An implementer who took the CL-01 wording at face value would either discover this at migration time and improvise, or - worse - implement the row-local half (the closed status list) and believe the cross-table half was also covered. This clarification removes the ambiguity at the source, and then applies the same discipline retroactively to every invariant in v0.1 and v0.2 whose enforcement was described loosely.

2. Exact proposed change - three mechanisms, three distinct scopes

The following definitions govern how enforcement language is to be read in v0.1, v0.2 and this amendment. Where any earlier text says "CHECK (or deferred trigger)", "write-path enforcement" or "consistency check" for a rule that spans rows or tables, it is to be read as the mechanism this section assigns - this is a clarification of enforcement, not a change to any rule itself.

Mechanism	What it can inspect	Correct use in this design
CHECK constraint	Values within the one row being written, and nothing else. Evaluated immediately, row by row, as the statement executes.	Closed value lists (*overall_status*, states, kinds); state-to-value rules pairing a column with its state column in the same row; same-row sign rules and arithmetic such as CL-16's variance identity; XOR rules over a row's own reference columns (CL-04, CL-16). If a proposed CHECK mentions another table, another row or a total, it is not a CHECK.
Deferred constraint trigger	Arbitrary queries over any tables, evaluated at COMMIT when declared DEFERRABLE INITIALLY DEFERRED - by which time every row of the transaction exists, in whatever order the transaction wrote them.	The enforcement mechanism for every cross-row and cross-table invariant in this design, including CL-01's matched-over-live-finding rule. Fires on each table whose writes can break the invariant, so the order of inserts inside the transaction does not matter.
Transaction-level assertion	Not a PostgreSQL feature. PostgreSQL does not implement the SQL-standard CREATE ASSERTION, and no syntax in this specification may imply that it does.	In this specification the phrase means exactly one of two real things, and documents must say which: a deferred constraint trigger as above, or a mandatory validation step inside the writing transaction, backstopped by such a trigger. It never means a CHECK, and it never means an application convention with no database backstop for a financial invariant.

CL-01 enforcement, restated correctly. The closed five-value list on *overall_status* remains a CHECK - it is row-local. The consistency rule is enforced by a deferred constraint trigger (ADAPT), defined on both *reconciliation_results* and *reconciliation_findings*, so that it fires whether a transaction writes the result first and its findings second, or the reverse. At commit, the trigger re-derives the expected overall status for each affected result using the CL-01 five-step precedence, reading the run's tolerance through the result's *run_id*, and rejects the transaction if the stored *overall_status* differs from the derived one. Verifying the whole derivation, rather than only the single "matched over a live finding" combination named in v0.2, costs nothing extra at commit time and closes the entire class of status errors, not one instance of it. Because results and findings are append-only, the trigger constrains creation only; there is no update path to police.

A note on trigger functions and RLS. Constraint-trigger functions execute with the privileges of the function owner, which in this design is the migrator role, not the runtime role - and under FORCE RLS the owner is itself subject to policies on row operations. Two design rules follow, stated so the implementation does not rediscover them: a trigger function used for cross-table verification must scope every query it makes by the *merchant_id* of the rows that fired it, explicitly, rather than relying on the caller's tenant context, which is a property of the application transaction and not of the verification being performed; and the existence of an owner-privileged trigger function is never a reason to grant the runtime role anything - triggers fire on writes the role is already permitted to make, and confer no callable surface of their own.

2. Exact proposed change (continued) - the mechanism map

Every cross-row or cross-table invariant named in v0.1, v0.2 or this amendment is assigned its mechanism here. This table is the complete list known to this review; a future invariant that cannot be placed in it is not yet fully specified.

Invariant	Mechanism	Source
Overall status equals the CL-01 derivation from findings and tolerance	Deferred constraint trigger (results + findings)	CL-01, as corrected here
Result totals equal the sum of their fee lines per basis, or are NULL	Deferred constraint trigger	v0.1 Section C; CL-08
A reversal's amount equals its target allocation's amount	Deferred constraint trigger	CL-10
Net allocations from one receipt never exceed the receipt amount	Row lock in the allocation transaction + deferred constraint trigger (mandatory)	v0.1 D.10 / OD-14; CL-10
At most one currently confirmed candidate per statement line	Deferred constraint trigger	CL-03; CL-13 item (f)
Match decision chain well-formedness for histories written through the application path	Advisory lock + partial unique index; deferred constraint trigger verification	CL-13
Variance calculation snapshots agree with their listed input set, under complete preconditions	Batch row lock in the computing transaction + deferred constraint trigger	CL-16
At least one active owner per workspace	Mandatory validation in the membership transaction, under lock	v0.1 OD-11 - no trigger backstop is specified there; flagged below
Closed value lists, state-to-value pairs, sign rules, same-row arithmetic, XOR reference rules	CHECK	Throughout Sections C and D
Tenant consistency of relationships	Composite foreign keys - not triggers and not CHECKs	v0.1 Sections C and E

One honest flag from the map. OD-11 (at least one active owner) is the only financial-adjacent invariant in the map whose enforcement rests on application transaction discipline alone, because v0.1 deliberately left the choice between a locking transaction and a deferred trigger open. This clarification does not close OD-11 - it is an open decision, and choosing its mechanism is part of it - but it narrows the choice honestly: if the decision lands on the trigger, the mechanism is a deferred constraint trigger as defined here; no third option exists.

3. Affected section and table

v0.2 CL-01 item 4 (the phrase "consistency CHECK (or deferred constraint trigger, ADAPT)" is superseded by section 2 above; the CL-01 *rule* - scenario-only V_{total} yields *open*, never *matched* and never a contractual *mismatch* - is untouched); v0.1 Section H.5 (the sentence assigning "the identities between a result's stored variances" to the database is to be read through the mechanism map); Section I test T-29 (new). No table, column or constraint list changes anywhere: this item changes what the specification *claims about* enforcement, not what is enforced.

4. Constraint or transaction-level enforcement

The enforcement is this clarification itself, plus one standing drafting rule for all future amendments and for the implementation review: **any sentence that assigns an invariant to "a CHECK" must be able to name the single table and the single row the check inspects.** If it cannot, the sentence is wrong and must name a deferred constraint trigger, a foreign key, a unique index, or a locked transaction protocol instead. Reviewers are asked to apply that test to this amendment as well: the CHECKs claimed in CL-14 (key state-to-column consistency), CL-15 (none claimed across tables - eligibility is a locked transaction rule, and section 4 of CL-15 says so) and CL-16 (same-row variance arithmetic; XOR reference rule) each pass it, and each of those findings assigns its cross-table parts to triggers or locks explicitly.

A performance note, so the trigger-based design is chosen with open eyes: deferred constraint triggers fire per affected row at commit, and the status trigger re-derives a result from its findings - work proportional to findings per result, at SME statement volumes that v0.1 Section F already takes as the design scale. This is not premature optimisation territory, and no caching or denormalisation is introduced to avoid it.

5. Updated test scenario

T-29 (new) - Cross-table enforcement mechanism. (a) Attempt, as a design-review assertion recorded with the test suite, to define the matched-over-live-finding rule as a CHECK referencing *reconciliation_findings*: the migration defining it is expected to be rejected by PostgreSQL, and the review records that rejection as the demonstration that CL-17's premise is correct. (b) In one transaction, insert a result with *overall_status* = matched and then a finding on it outside tolerance: the transaction fails at commit, not at either insert. (c) Reverse the write order - findings first, result second - with the same inconsistency: the transaction likewise fails at commit. (d) A result whose findings are all within tolerance, stored as *matched_with_tolerance*: commits. (e) A scenario-only V_{total} finding outside tolerance stored with overall *open*: commits; the same rows stored with overall *matched* or *mismatch*: rejected at commit - CL-01 and CL-17 verified together. (f) A row violating the closed status list (a sixth value): rejected immediately at insert by the CHECK, demonstrating the division of labour in a single scenario.

6. Status

Resolved as a clarification. No rule changes: CL-01 stands exactly as v0.2 decided it. What changes is that the specification no longer claims an enforcement mechanism PostgreSQL does not have, every previously loose enforcement phrase has an assigned real mechanism, and a drafting test exists to keep the error from returning. This is the only item in the amendment that corrects v0.2's own wording, and it is recorded in the change log as such.

Consolidated test changes

Section I of v0.1, as amended by v0.2, stands except as listed here. This table is the complete delta from this amendment.

Test	Change	Source
T-17	Strengthened: the phrase "the historical computation remains auditable" now asserts the immutable calculation row (seq 1) and its typed input references, not an audit payload	CL-16
T-25	Revised: Stage 2 tombstoning assertions - identity fields scrubbed in place, financial rows byte-identical including merchant_id, runtime role reads nothing, re-open rejected after anonymisation	CL-15
T-26	New: match decision chain integrity - serialised concurrency, single-successor index, same-candidate composite FK, conflicted derivation, unanimous-rejection closure, successor candidate	CL-13
T-27	New: idempotency recovery - crash windows W1 to W4, lease reclaim, zombie fencing, reclaim race, hash conflict in every state, attempt cap, post-expiry run deduplication	CL-14
T-28	New: bank variance history - calculation sequence, input sets, current-variance derivation, supersession after reversal, append-only enforcement	CL-16
T-29	New: cross-table enforcement - CHECK incapacity demonstrated, commit-time rejection, CL-01 derivation enforced by the deferred trigger	CL-17
T-30	New: shared user across workspaces - one workspace's anonymisation never touches a user with an active membership elsewhere; global anonymisation only when no active membership remains	CL-15

Appendix 1 delta - invariant traceability

- **Row "Corrections create new records or runs"** now also covers match decision chains (CL-13), idempotency claims (CL-14) and bank variance calculations (CL-16). Tests T-21, T-26, T-28.
- **Row "Bank variance NULL until receipt complete and batch fully allocated"** - enforcement now reads: calculation insert preconditions and derived current value over *settlement_variance_calculations* (CL-16); tests T-17, T-28. The invariant itself is unchanged.
- **Row "Scenario V_total cannot trigger a contractual mismatch"** - enforcement now reads: D.6 derivation as corrected by CL-01, verified at commit by the deferred trigger assigned in CL-17; tests T-16, T-29.
- **New row: match decisions form one linear chain per candidate** - advisory-lock serialisation, partial unique successor index, same-candidate composite self-FK (CL-13); test T-26.
- **New row: a claimed idempotency key always reaches a defined end state** - lease expiry, fenced reclaim, same-transaction completion (CL-14); tests T-10, T-27.
- **New row: anonymisation changes identity, never keys** - in-place tombstoning under ON DELETE RESTRICT, with no financial-row writes (CL-15); tests T-25, T-30.

> No test is claimed to have passed. These are design-stage scenarios for the future M1 test suite, as in v0.1 Section I and v0.2. Nothing in this amendment was executed against a database, because no M1 implementation exists and none is authorised.

Change log - v0.2 to v0.3 (this amendment)

Item	Type	Change
CL-13	Supersedes + Adds	Decision history defined as one linear chain per candidate: advisory-lock serialisation of decision inserts; partial unique index for at most one successor per decision; composite self-FK confining supersession to the same candidate and tenant; deterministic derivation for well-formed, unanimously rejected and conflicted histories; conflict resolution by closure or by a successor candidate (new *prior_candidate_id* column on *shipment_match_candidates*); cycle impossibility shown from immutability; T-26 added.
CL-14	Clarifies + Adds	Idempotency recovery specified on the CL-04 architecture: lease expiry separated from key expiry; fencing *claim_token*, *attempt_count* and *claimed_by_instance* columns added; atomic conditional reclaim; completion made explicitly the last write of the work transaction, closing crash window W3; four crash windows stated; attempt cap defined; T-27 added, T-10 extended.
CL-15	Clarifies + Adds	CL-12 Stage 2 given mechanics: tombstone merchant anonymised in place (*anonymised_at* added), financial rows untouched, shared retention tenant and merchant_id rewriting considered and rejected; global-eligibility rule for user anonymisation protecting shared accounts; post-anonymisation RLS and export behaviour defined; T-25 revised, T-30 added; OD-17 opened for personal data inside append-only raw records.
CL-16	Supersedes + Adds	Bank variance history moved from the single stored column on *settlement_batches* (removed) to two new append-only tables, *settlement_variance_calculations* and *settlement_variance_calculation_inputs* (31 tables in total); current variance defined as a derivation over the latest calculation and its still-holding preconditions; CL-10's audit-record wording corrected to a mirror relationship; T-28 added, T-17 strengthened.
CL-17	Clarifies (corrects v0.2 wording)	CL-01 item 4's "consistency CHECK (or deferred constraint trigger)" superseded: the cross-table rule is a deferred constraint trigger, full stop; CHECK, deferred constraint trigger and "transaction-level assertion" defined with their real scopes; a mechanism map assigns every cross-row invariant in v0.1 and v0.2; a standing drafting rule for future enforcement claims; OD-11 narrowed but not closed; T-29 added. No rule in CL-01 to CL-12 is changed by this item.

Types follow v0.2 usage: *Supersedes* - the earlier text named is replaced for that item only. *Clarifies* - the earlier text was ambiguous or incomplete; one reading is fixed and the other rejected explicitly. *Adds* - new material with no direct counterpart.

Remaining blockers

Blocking any M1 implementation (unchanged from v0.1 and v0.2)

- **B-1.** Market validation: Day 7 and Day 14 gates not yet run; no interview evidence exists.
- **B-2.** No merchant-confirmed courier evidence for any of Steadfast, Pathao, Paperfly, RedX or eCourier - no rule version can yet be populated without inventing contract semantics, which is prohibited.
- **B-3.** Docker + PostgreSQL verification environment not established; no design SQL in v0.1, v0.2 or this amendment has been executed - including the deferred triggers, advisory-lock protocol and conditional-update patterns specified here, which are design-reviewed but unproven against a running database.
- **B-4.** Explicit production-readiness approval not given.

Open decisions carried forward

- **OD-12 (CL-02, v0.2)** - whether any real courier statement reports a negative net payout. Open until redacted merchant statements answer it. Unchanged by this amendment; CL-16's negative bank *variance* is a different, purely arithmetic quantity and is not evidence for OD-12 either way.
- **OD-16 (CL-12, v0.2)** - workspace grace-window length, Bangladesh financial retention period, and the erasure-versus-retention conflict rule. Requires policy and legal review. CL-15 supplies the mechanics OD-16 was missing; the periods and the conflict rule remain undecided. Blocks production launch, not design review.
- **OD-17 (new, CL-15)** - treatment of third-party personal data held inside append-only raw records (*import_rows.raw_values*, statement raw fields) when a workspace is anonymised, where erasure, raw-preservation immutability and financial retention conflict. Interim position: sealed retention, excluded from anonymised exports, no runtime-role access. Requires the same Bangladesh legal review as OD-16. Blocks production launch, not design review.
- **OD-11 (v0.1, narrowed by CL-17)** - guaranteeing at least one active owner per workspace: mechanism choice is now between a locked membership transaction and a deferred constraint trigger, and the decision itself remains open.
- **CL-11 deployment check (v0.2)** - the tenant-context trust boundary is specified; verifying a real deployment honours it remains a production-readiness task.
- All other v0.1 open decisions OD-01 to OD-15 stand as written, except OD-11 and OD-12 as noted above; OD-13's expiry window is to be read together with the 24-hour design default set by v0.2 CL-04.

> **Final statement.** With this amendment applied, the five audited items are settled at design level: decision histories cannot branch undetectably (CL-13), abandoned idempotency claims always recover without duplicating a financial operation (CL-14), anonymisation no longer threatens tenant integrity or shared accounts (CL-15), bank variance history is structured and immutable (CL-16), and no enforcement claim in the specification exceeds what PostgreSQL can actually do (CL-17). Three things remain open for named external reasons - evidence for OD-12, and Bangladesh legal and policy review for OD-16 and OD-17 - rather than design ambiguity. The specification, v0.1 as amended by v0.2 and v0.3, is a coherent, reviewable blueprint. It is not an authorisation to build: M1 production implementation remains gated behind market validation and explicit production-readiness approval.

End of amendment v0.3. This amendment supersedes only the sections it explicitly names. All other content of the Database Design Specification v0.1 and of Technical Correction Amendment v0.2 remains unchanged.